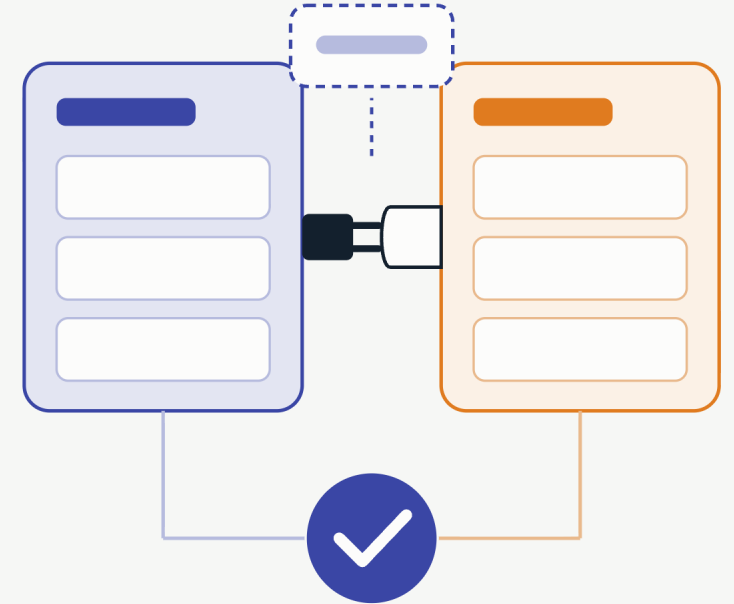


Интеграција модула, уговори и подаци

EquipmentReservation као извршива основа за интеграцију модула и наставак развоја.



Садржај презентације

01

Solution и границе

Покретање, смер
зависности и
одговорности
пројеката.

04–07

02

Правило и валидација

Правило залихе у
доменском сервису и
валидатор команде.

08–10

03

Уговор између модула

Порт, handler,
idempotency key
и тест.

11–17

04

Докази и ризици

Build, тестови, diff,
API граница и AI
радни ток.

18–23

05

Вежба и припрема

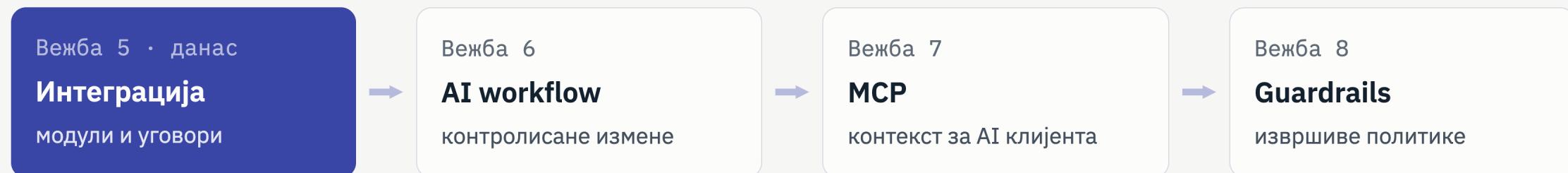
Задатак на вежби,
резиме и припрема.

24–27

Увод · заједнички извршиви пример (03)

Заједнички извршиви пример

EquipmentReservation се надограђује кроз интеграцију, AI радни ток, MCP и guardrails.



НАПОМЕНА

EquipmentReservation.sln је главна улазна тачка наставног примера.

01

ПРВИ ДЕО

Solution и границе

Покретање примера и провера да ли пројекти зависе у добром смеру.

restore, build, test

Смер зависности

Одговорности пројеката

Покретање и провера наставног примера

Прва провера обухвата `restore`, `build` и `test` команде над `solution`-ом.

terminal

```
1  cd examples/ers-ai-workflow
2  dotnet restore EquipmentReservation.sln
3  dotnet build EquipmentReservation.sln -c Release --no-restore
4  dotnet test EquipmentReservation.sln -c Release --no-build
```

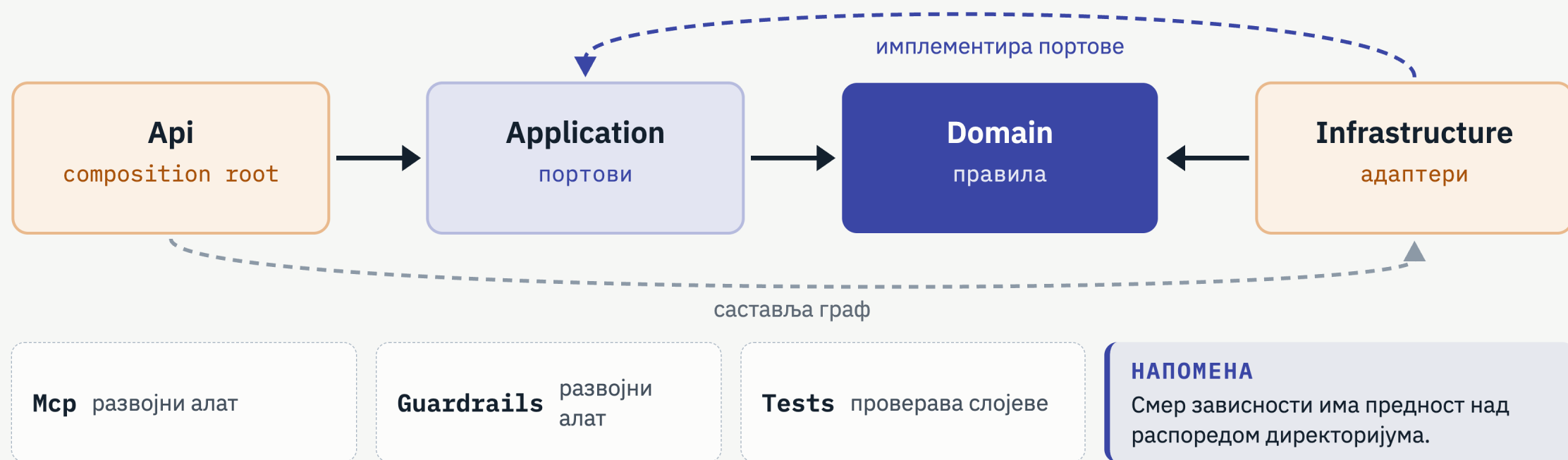
Радимо над целим `solution`-ом.

Команде су део доказа, а не декоративан додатак.

Пад `build`-а или теста мора се решити пре даље интеграције.

Структура solution-а и смер зависности

MCP и Guardrails припадају истом solution-у, без увођења зависности у пословно језгро.



Одговорности пројеката у solution-у

Сваки пројекат има јасно дефинисану одговорност.

Пројекат	Одговорност
Domain	ентитети и пословна правила, без пројектних зависности
Application	use-case-ови и портови према другим модулима
Infrastructure	имплементације портова
Api	composition root и HTTP граница
Mcp	контролисано излагање контекста AI клијенту
Guardrails	детерминистичке политике за ризичне позиве
Tests	NUnit провере домена, use-case-a и guardrail-a

02

ДРУГИ ДЕО

Правило и валидација

Где живи правило о залихи и шта валидатор проверава пре пословног тока.

InventoryItem

InventoryReservationService

Валидатор команде

Пословно правило у доменском сервису

InventoryItem представља стање и нема зависности од HTTP-а, базе, MCP-а или AI алата.

InventoryReservationService: количина мора бити позитивна и не сме бити већа од расположиве количине.

ПРИМЕР • РАСПОЛОЖИВО 5 КОМАДА

захтев 3  дозвољено, остају 2

захтев 6  одбијено, више од расположивог

захтев 0  одбијено, количина мора бити позитивна

НАПОМЕНА

Исто правило користе API, тестови и будући адаптери без дуплирања.

Валидација захтева

Валидатор проверава облик и опсег улаза пре покретања пословног тока.

C# • Application

```
1 public Result Validate(CreateReservationCommand command)
2 {
3     ArgumentNullException.ThrowIfNull(command);
4     if (command.RequestId == Guid.Empty)
5         return Result.Fail(
6             CreateReservationErrorCodes.RequestIdRequired);
7     if (command.Quantity <= 0)
8         return Result.Fail(
9             CreateReservationErrorCodes.QuantityMustBePositive);
10    return Result.Ok();
11 }
```

НАПОМЕНЕ

Модел само носи стање.

Validator проверава обавезна поља и опсег.

Handler покреће проверу пре позива портова.

03

ТРЕЋИ ДЕО

Уговор између модула

Како Reservations користи Inventory без увида у његове интерне детаље.

IInventoryModule

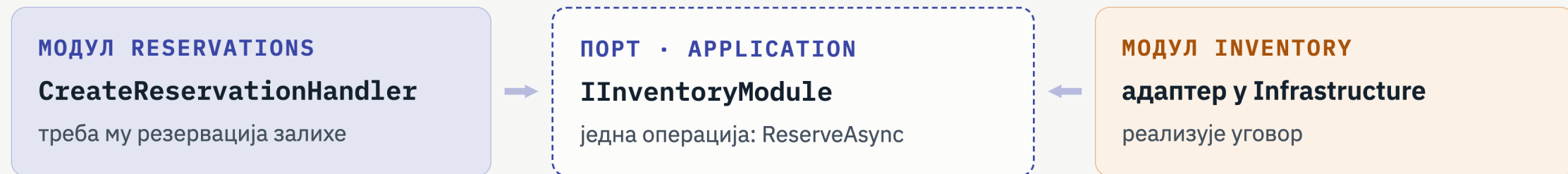
CreateReservationHandler

Idempotency key

Тест идемпотентности

Уговор између модула Reservations и Inventory

Апликациони слој описује минималне операције потребне случају употребе.



УГОВОР НЕ СМЕ ДА

проследи ORM ентитет

отвори интерно складиште

прекрши границу другог модула

НАПОМЕНА

Уговор између модула примењује ISP и DIP.

Порт апликационог слоја

Случај употребе зависи од минималног уговора који му је потребан.

```
C# • Application
1  public sealed record ReserveInventoryRequest(
2      Guid EquipmentId,
3      int Quantity,
4      Guid ReservationId);
5
6  public interface IInventoryModule
7  {
8      Task<Result> ReserveAsync(
9          ReserveInventoryRequest request,
10         CancellationToken cancellationToken);
11 }
```

НАПОМЕНЕ

Порт је у Application слоју.

Infrastructure реализује уговор.

Use-case не зна интерне детаље Inventory модула.

Handler случаја употребе • 1/2

Handler валидира улаз и проверава да ли је исти захтев већ обрађен.

```
C# • CreateReservationHandler
1  var validation = _validator.Validate(command);
2  if (!validation.Success)
3      return CreateReservationResult.Invalid(validation.Error);
4
5  await using var lease = await _requestLock
6      .AcquireAsync(command.RequestId, ct);
7  var existing = await _reservations
8      .FindByRequestIdAsync(command.RequestId, ct);
9  if (existing is not null)
10     return CreateReservationResult.From(existing, replayed: true);
```

КОРАК ПО КОРАК

1–3: невалидан улаз не стиже до порта.

5–6: исти RequestId се не обрађује истовремено.

7–10: поновљени захтев враћа постојећи исход.

Handler случаја употребе • 2/2

Handler позива Inventory преко порта и бележи потврђену или одбијену резервацију.

C# • CreateReservationHandler • наставак

```
12 var reservationId = _idGenerator.NewReservationId();
13 var inventoryResult = await _inventory.ReserveAsync(
14     new(command.EquipmentId, command.Quantity, reservationId), ct);
15 var reservation = inventoryResult.Success
16     ? Reservation.Confirmed(reservationId, command.RequestId,
17         command.EquipmentId, command.StudentId, command.Quantity)
18     : Reservation.Rejected(reservationId, command.RequestId,
19         command.EquipmentId, command.StudentId, command.Quantity,
20         inventoryResult.Error);
21 await _reservations.AddAsync(reservation, ct);
22 return CreateReservationResult.From(reservation, replayed: false);
```

НАПОМЕНЕ

Модел нема пословне методе, само factory за стања.

Inventory правило остаје иза порта.

Application враћа стабилан исход API слоју.

Idempotency key и поновљени захтев

Поновљени захтев са истим идентификатором не производи додатни ефекат.

RequestId | један захтев, један ефекат

1. позив

нема резервације за RequestId



Inventory.ReserveAsync



Confirmed · replayed: false

2. исти позив

резервација већ постоји



Inventory се не позива



иста резервација · replayed: true

Тест идемпотентности

Тест потврђује да поновљени позив не мења стање други пут.

C# • NUnit

```
1 var first = await handler.HandleAsync(command, ct);
2 var replay = await handler.HandleAsync(command, ct);
3 var available = await inventory.GetAvailableAsync(
4     command.EquipmentId, ct);
5
6 Assert.That(first.Outcome,
7     Is.EqualTo(CreateReservationOutcome.Confirmed));
8 Assert.That(replay.ReservationId,
9     Is.EqualTo(first.ReservationId));
10 Assert.That(replay.Replayed, Is.True);
11 Assert.That(available, Is.EqualTo(3));
```

НАПОМЕНЕ

Тест проверава уговор између модула.

Поновљени позив не сме да направи нови side effect.

Проверавају се и исход и преостала количина.

04

ЧЕТВРТИ ДЕО

Докази и ризици

Шта потврђује да је интеграција исправна и где најчешће пуца.

Build, Tests, Diff, Contract

API граница

Ризици

Заједнички производ

AI радни ток

Докази интеграције

Build, тестови, diff и уговори потврђују исправност интеграције.

01



Build

целокупан solution се
компајлира

02



Tests

пословни и интеграциони
сценарији пролазе

03



Diff

промена је ограничена на
очекиване слојеве

04



Contract

уговор између модула
је стабилан

API kao composition root

HTTP граница саставља зависности и мапира резултат без пословних одлука.

API ради

прима HTTP захтев

мапира DTO у команду

позива handler

мапира Result у одговор

API не ради

не проверава залиху директно

не мења Domain стање мимо use-case-a

не чита интерну базу другог модула

не садржи пословну одлуку

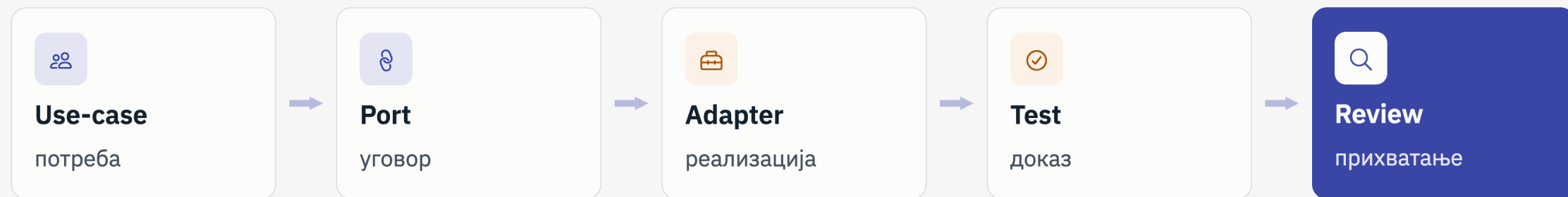
Ризици интеграције

Сваки ризик повезује се са одговарајућом провером.

Ризик	Провера
Погрешан смер зависности	project references и build
Дуплирана правила у API-ју	code review и diff
Поновљен захтев прави нови ефекат	idempotency тест
Порт пропушта интерне детаље	преглед Application уговора
MCP или Guardrails улазе у Domain	провера dependency rule-a

Интеграција модула у заједнички производ

Модули сарађују преко уговора, адаптера, тестова и прегледа кода.



НАПОМЕНА

Модули се повезују преко стабилних уговора са јасним пословним значењем.

Границе модула и AI радни ток

Стабилне границе ограничавају контекст, обим измене и поступак провере.

01



Контекст

погођени пројекти су познати

02



Ограничења

зна се шта не сме да се мења

03



Провера

build, test и diff су довољни сигнали

НАПОМЕНА

Што су границе јасније, то је AI задатак мањи и лакши за проверу.

05

ПЕТИ ДЕО

Вежба и припрема

Рад на заједничком примеру и припрема за контролисан AI радни ток.

Задатак на вежби

Резиме

Припрема

Задатак на вежби

EquipmentReservation се користи као заједнички извршиви пример.

- 01 Отворити EquipmentReservation.sln.
- 02 Проверити build и test команде.
- 03 Пронаћи Domain правило и његов Application порт.
- 04 Објаснити како CreateReservationHandler користи Inventory.
- 05 Показати тест који штити идемпотентност или уговор између модула.

Интеграција повезује модуле преко стабилних уговора уз очуване архитектонске границе.

Добра интеграција значи да модули сарађују преко јасних уговора, док пословно језгро остаје заштићено од детаља транспорта, складишта и развојних алата.



Наредна вежба уводи контролисан AI радни ток.

Припрема за наредну вежбу

Припремити solution, тестове и границе модула за контролисан AI радни ток.

01

Забележити како се solution покреће и тестира.

02

Описати границе Domain, Application, Infrastructure и API пројеката.

03

Изабрати један мали захтев за промену правила.

04

Навести фајлове које промена сме да погађа.

05

Припремити проверу build, test и diff за следећу вежбу.